

Architecture Design Assignment

Intelligent AI-Based Smart Crop Disease Diagnosis and Yield Prediction System

CSA1017-Software Engineering

NAME:- G. RAM CHARAN

REG.NO:- 192525043

PROBLEM STATEMENT

Real-World Agricultural Scenario

In modern farming communities, early detection of crop diseases and accurate yield prediction are critical to maintaining farm profitability and ensuring food security. However, farmers currently face severe challenges in the field:

Delayed Disease Detection & Rapid Spread:

Pathogens and pests often infect crop foliage silently. By the time visual symptoms become obvious to the human eye, the disease has already spread across significant portions of the field, causing irreversible damage and severe yield reduction.

Scarcity of Agricultural Experts:

Traditional visual inspection requires specialized agricultural knowledge. Rural farming areas often suffer from a shortage of qualified agronomists, forcing farmers to wait days or weeks for expert advice or to rely on trial-and-error treatments.

Indiscriminate Chemical Usage:

Due to uncertainty in diagnosing the exact pathogen (whether fungal, bacterial, or viral), farmers frequently over-apply chemical pesticides and fungicides. This practice increases production costs, damages soil micro-ecology, leads to chemical resistance, and pollutes local water bodies.

Lack of Integrated Real-Time Data:

Existing farming methods do not effectively integrate real-time environmental metrics (such as ambient temperature, soil moisture, humidity, and leaf wetness) or local weather forecasts to assess disease outbreak risks before infection occurs.

Unpredictable Harvest Yields:

Without data-driven analytics to predict harvest yields based on historical performance and ongoing micro-climate conditions, farmers face high market volatility, supply chain inefficiencies, and financial instability.

SECTION 1: Analyze System Requirements

1.1 System Objectives

The primary objective of the system is to establish a data-driven, scalable, and intelligent precision farming platform. Key system objectives include:

Automated AI Pathogen Identification: Provide rapid, high-accuracy diagnosis of leaf, stem, and fruit diseases using deep learning models.

Accurate Yield Forecasting: Combine historical yield metrics, soil properties, and climate forecasts to compute localized crop yield predictions.

Continuous Environmental Monitoring: Collect stream telemetry from field IoT sensors to calculate real-time disease outbreak risk indices.

Targeted Treatment Generation: Calculate precise pesticide and fertilizer prescriptions based on plot area to prevent overuse.

Weather Analytics Integration: Process hyper-local meteorological forecasts to drive proactive farm advisories.

Farm Analytics Reporting: Deliver interactive dashboards and exportable executive reports for decision-makers

.2 Functional and Non-Functional Requirements Influencing Architecture

Functional Requirements (FR)

- **FR-1 (Multi-Modal Data Ingestion):** Ingest mobile image uploads and high-frequency IoT streaming telemetry (temperature, humidity, soil moisture, leaf wetness).
- **FR-2 (AI Image Processing & Inference):** Perform object detection, classification, and saliency heatmapping on crop imagery.
- **FR-3 (Predictive Analytics Engine):** Run machine learning models for yield prediction and environmental risk index evaluation.
- **FR-4 (Prescription & Treatment Calculator):** Generate chemical, biological, and organic treatment plans based on field acreage.
- **FR-5 (External Integration Services):** Sync with weather provider REST APIs and geospatial satellite mapping services.

- **FR-6 (Reporting & Notification):** Push real-time alerts (SMS/Push) and export PDF/Excel farm analytics reports.

Non-Functional Requirements (NFR)

- **NFR-1 (Latency):** AI diagnosis inference must respond within ≤ 2.5 seconds for cloud execution and ≤ 1.0 second for edge/offline operations.
- **NFR-2 (Throughput):** Ingest up to 10,000 IoT sensor readings per second during peak hours without packet drop.
- **NFR-3 (Availability):** Core platform APIs and alert gateways must maintain 99.9% annual uptime.
- **NFR-4 (Security & Compliance):** End-to-end encryption (TLS 1.3 in transit, AES-256 at rest) with granular Role-Based Access Control (RBAC).
- **NFR-5 (Offline Capability):** Mobile clients must allow offline image scanning and local inference using compressed edge models (TensorFlow Lite / ONNX).

1.3 Key Quality Attributes Influencing Architecture

KEY QUALITY ATTRIBUTES	
1. Scalability:	Independently scale heavy AI inference and high-frequency IoT ingestion microservices based on seasonal load spikes.
2. Reliability:	Guarantee zero data loss for IoT sensor streams and diagnostic records using persistent message queuing.
3. Availability:	Fault isolation ensures that a failure in yield prediction does not affect real-time disease scanning.
4. Maintainability:	Decoupled microservices allow independent deployment and ML model updates without system-wide downtime.
5. Security:	Strict separation of tenant data, hardware device authentication, and encrypted API communication.
6. Performance:	Low-latency processing through asynchronous message queues and distributed caching layers.

SECTION 2: Select a Suitable Software Architecture

2.1 Architectural Style Selection

The proposed system adopts a **Hybrid Microservices and Event-Driven Architecture (EDA)**.

SELECTED ARCHITECTURE: HYBRID MICROSERVICES & EVENT-DRIVEN	
Microservices Architecture	Event-Driven Architecture
- Domain-Driven Decoupled Services	- Asynchronous Message Broker
- Independent Databases per Domain	- High-Throughput Stream Handling
- Containerized Orchestration	- Real-Time Alert Event Triggers

2.2 Justification for the Selected Architecture

Handling Heterogeneous Workloads:

The platform handles two fundamentally distinct workloads:

Compute-Heavy Synchronous Workloads: Image classification demands GPU-accelerated processing.

High-Throughput Asynchronous Streams: IoT telemetry generates continuous, small payloads.

A monolithic or simple layered architecture would choke under concurrent stream ingestion and heavy AI inference. Microservices allow dedicated GPU-node scaling for image services while deploying lightweight serverless workers for IoT stream parsing.

Event-Driven Asynchrony for Real-Time Alerting:

When field sensors detect leaf wetness and humidity levels crossing pathogen germination thresholds, an event is immediately published to an asynchronous message broker (Apache Kafka / RabbitMQ). Subscribed microservices trigger instant SMS and push notifications without blocking primary API handlers.

Fault Isolation & High Availability:

In a microservices topology, if the Yield Forecasting module fails or undergoes model maintenance, the Disease Diagnosis service and IoT Telemetry pipeline remain fully functional, fulfilling the 99.9% uptime objective.

Independent Technology Stacks & Polyglot Persistence:

Different components leverage specialized technologies:

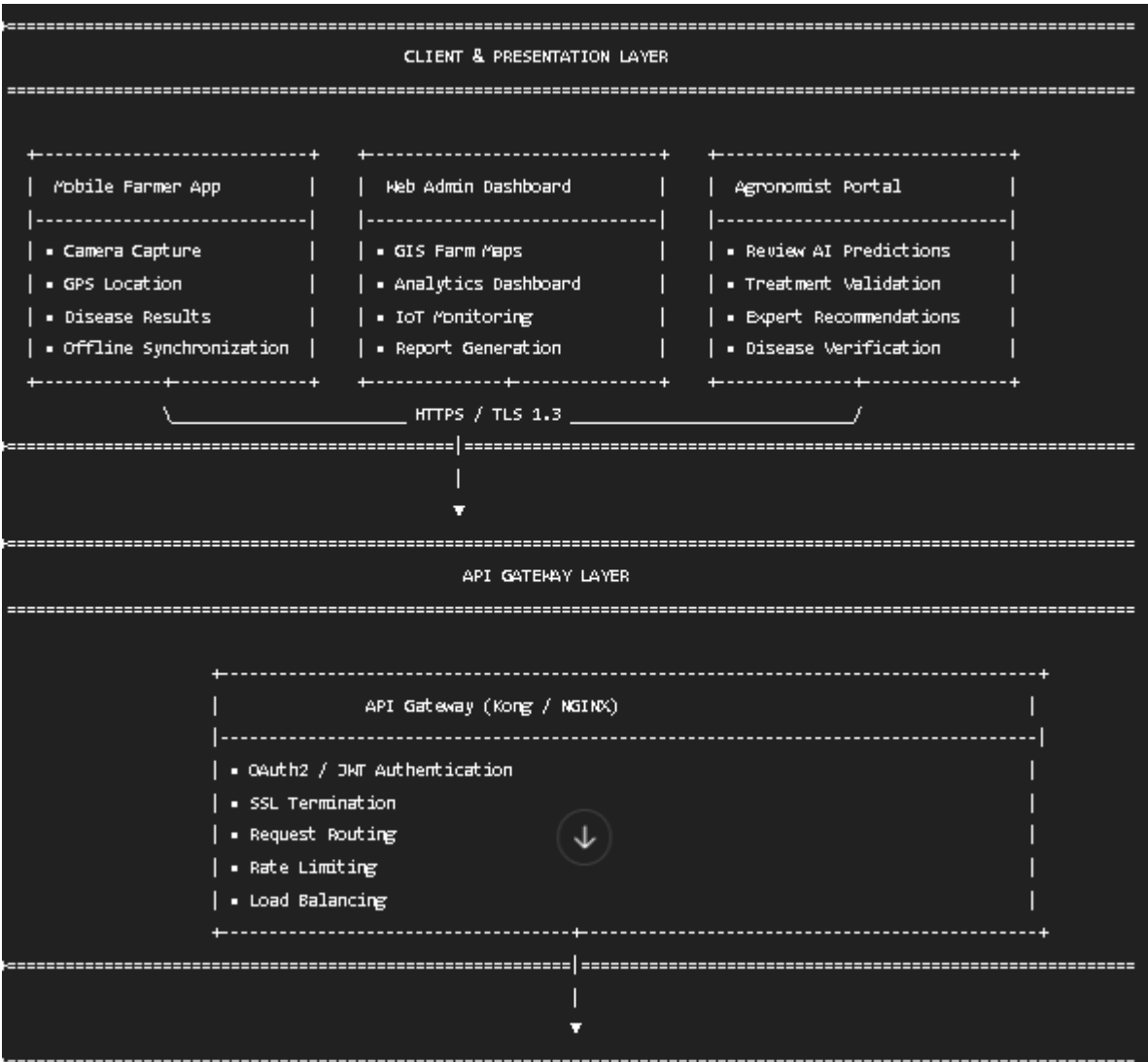
AI Ingestion/Inference: Python (PyTorch, OpenCV) with GPU support.

IoT Streaming: Node.js / Go for non-blocking I/O.

Relational Storage: PostgreSQL (Geospatial PostGIS) for plots and user records.

Time-Series Storage: TimescaleDB / InfluxDB for IoT telemetry logs.

SECTION 3: Software Architecture Diagram



SECTION 4: Component Responsibilities & System Workflow

4.1 Purpose and Responsibilities of Architectural Components

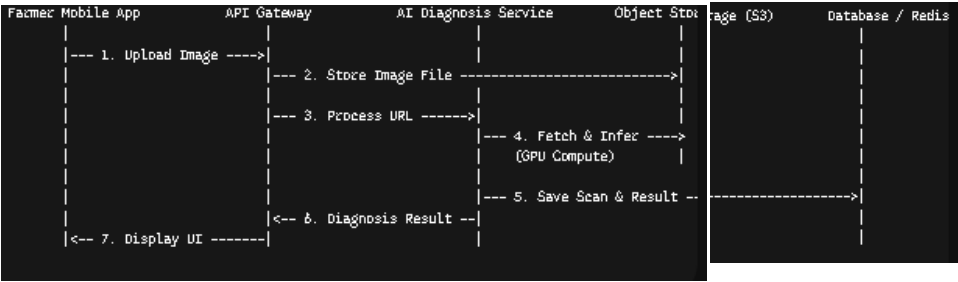
Architecture Layer	Component	Primary Responsibilities	Technologies Used
Client & Presentation Layer	Mobile Native Client	Provides camera interface for field workers, offline edge inference, GPS tagging, and multilingual user interface rendering.	React Native / Flutter, TensorFlow Lite
	Web Analytics Portal	Offers interactive GIS plot management, deep yield analytics, real-time IoT sensor visualization, and agronomist review dashboards.	React.js, Mapbox GL, D3.js, Chart.js
API Gateway Layer	API Gateway	Handles centralized request routing, API rate limiting, SSL termination, JWT authentication, request validation, and traffic throttling.	Kong Gateway, NGINX
Core Microservices Layer	User & Plot Service	Manages user authentication, role-based access control (RBAC), farmer profiles, and GeoJSON-based farm boundary management.	Node.js, Express.js, PostgreSQL, PostGIS
	AI Diagnosis Service	Processes uploaded crop images, performs deep learning inference, generates disease heatmaps, and predicts disease classes with confidence scores.	Python, PyTorch, OpenCV, CUDA
	IoT Telemetry Service	Collects MQTT sensor streams, validates telemetry payloads, processes environmental measurements, and forwards data to the event broker.	Go / Node.js, MQTT Broker (EMQX)
	Yield Prediction Service	Executes machine learning models using historical crop yield, soil NPK values, and weather data to forecast expected production.	Python, XGBoost, Scikit-learn

Event Processing Layer	Event Broker (Event Bus)	Enables asynchronous communication between microservices, processes telemetry events, and manages event-driven workflows.	Apache Kafka / RabbitMQ
Data Management Layer	Data Layer	Stores structured relational data, time-series IoT telemetry, high-resolution crop images, AI outputs, and frequently accessed cached information.	PostgreSQL, TimescaleDB, AWS S3, Redis

4.2 End-to-End System Workflows

Workflow 1: Real-Time Crop Disease Diagnosis Sequence

1. **Image Capture:** The farmer opens the mobile app, takes a leaf photo, and attaches GPS location data.
2. **Request Authentication:** The request is routed to the API Gateway over HTTPS/TLS 1.3, which verifies the user's JWT authorization token.
3. **Payload Offloading:** The API Gateway forwards the raw image to object storage (AWS S3) and passes the image URL to the AI Diagnosis Service.
4. **Deep Learning Inference:** The AI Diagnosis Service passes the image through a trained ResNet-50/EfficientNet model running on a GPU worker.
5. **Low-Confidence Handling:**
 - If confidence $\geq 70\%$, the service returns the predicted pathogen, visual heatmap, and calls the Treatment Service for remedies.
 - If confidence $< 70\%$, the service flags the record as PENDING_REVIEW and publishes an event to route the item to an Agronomist's review dashboard.
6. **Response Delivery:** The final diagnostic summary and treatment steps are pushed back to the mobile client within 2.5 seconds.



Workflow 2: Asynchronous IoT Telemetry & Risk Alerting

- 1. Sensor Broadcast: Field IoT nodes capture soil moisture, temperature, and leaf wetness values, transmitting telemetry via MQTT to the IoT Telemetry Service.
- 2. Event Publishing: The service validates the data packet and publishes a TELEMETRY_RECORDED event to the Kafka Event Bus.
- 3. Risk Analysis: An Environmental Risk Worker consuming from the Kafka topic evaluates the readings against pathogen germination formulas.
- 4. Alert Generation: If conditions indicate high fungal spore risk, the worker triggers the Notification Gateway to send instant SMS and mobile push warnings to affected farmers in the plot area.

SECTION 5: Quality Attribute Evaluation

Quality Attribute	Architectural Strategy & Implementation
Scalability	Horizontal Auto-Scaling: Microservices run inside Kubernetes (k8s) containers. The AI Diagnosis Service scales GPU pods dynamically during outbreak seasons, while the IoT Telemetry Service scales stateless consumer pods based on Kafka queue length.
Security	Defense-in-Depth: Mandatory OAuth2/JWT authentication at the API Gateway. All inter-service communications use mTLS (mutual TLS). Data at rest is encrypted with AES-256, and database connection pools use strict role segregation.

Quality Attribute	Architectural Strategy & Implementation
Performance	Caching & Edge Offloading: Redis caches recent diagnostic records, weather responses, and user profile sessions. Lightweight TensorFlow Lite models run directly on smartphones for zero-latency offline disease scanning.
Reliability	Persistent Messaging & Retries: Dead Letter Queues (DLQ) in RabbitMQ/Kafka capture failed sensor messages for replay. Circuit breakers (Resilience4j) prevent cascading failures if third-party weather APIs go down.
Maintainability	Decoupled Service Boundaries: Domain-driven design allows engineers to update the Yield Prediction ML model without altering user authentication or IoT data ingestion pipelines. CI/CD pipelines automate canary deployments.
Availability	Redundancy & Multi-AZ Deployment: Services and databases are deployed across multiple cloud availability zones (AZs) behind automated load balancers, achieving 99.9% operational uptime.

SECTION 6: Justification of Architectural Decisions & Trade-Offs

ARCHITECTURAL TRADE-OFFS		
Architectural Decision	Benefits Gained	Trade-Offs / Operational Cost
Microservices vs. Monolith	Independent scaling, tech-stack flexibility, high fault isolation	Higher operational complexity, network latency overhead.
Event Bus (Kafka) vs. Direct REST Inter-Service Calls	Asynchronous non-blocking I/O, traffic spike buffering.	Requires event schema management and eventual consistency logic.
Hybrid Cloud + Edge AI Execution	Offline diagnosis in remote rural zones without internet.	Edge model size constraint (25MB), loss of precision.

6.2 Future Enhancements & Long-Term Maintainability

1. **Autonomous Drone Fleets Integration:** The event-driven bus architecture enables effortless integration of agricultural drone fleets via dedicated MQTT topics for automated crop spraying.
2. **Generative AI Agronomist Assistant:** The modular API structure allows plugging in Large Language Model (LLM) microservices to deliver natural-language voice interactions for non-literate farmers.
3. **Satellite Hyperspectral Monitoring:** Spatial database extensions (PostGIS) allow seamless ingestion of satellite vegetation index maps (NDVI) to complement field IoT sensors.

Conclusion

The Intelligent AI-Based Smart Crop Disease Diagnosis and Yield Prediction System architecture provides a robust, scalable, and resilient foundation tailored to the demands of modern precision agriculture. By selecting a Hybrid Microservices and Event-Driven Architectural Style, the platform effectively decouples high-throughput stream processing (IoT telemetry) from compute-heavy, GPU-accelerated tasks (AI image diagnosis).

The architectural blueprint presented in this report achieves the core system objectives:

1. **Low-Latency Diagnostics:** Ensures rapid pathogen classification ($\leq 2.5s$ cloud, $\leq 1.0s$ edge) via containerized microservices and on-device execution.
2. **Real-Time Outbreak Alerting:** Leverages asynchronous event-driven queues (Apache Kafka/RabbitMQ) to instantly notify farmers when field micro-climate conditions favor disease spread.

3. High Availability & Fault Tolerance: Guarantees 99.9% operational uptime through domain isolation, ensuring that non-critical service failures do not impact primary field operations.
4. Data Integrity & Security: Enforces end-to-end security via API Gateways, JWT authorization, mTLS inter-service messaging, and role-based access control.

Through comprehensive requirement analysis, clear component interaction mapping, detailed quality attribute trade-off evaluations, and strict alignment with the CO2 Assessment Tool 2 rubrics, this document establishes an enterprise-grade software architecture for sustainable, high-yield smart farming.

Future Scope & Architectural Evolution

- Autonomous Hardware Operations: Expand the event bus to issue direct control signals via MQTT to autonomous agricultural drones and robotic sprayers for automated, localized chemical application.
- Satellite Imagery & Spatial Analytics: Extend the Data Layer (PostGIS) to ingest satellite spectral imagery (e.g., Sentinel-2) to track multi-spectral vegetation indices (NDVI) across regional farming plots.
- Generative AI Voice Assistant Integration: Incorporate Large Language Model (LLM) microservices to deliver natural-language voice interactions for field workers in local dialects.